



UNIVERSIDADE
FEDERAL DO CEARÁ
CAMPUS QUIXADÁ

Gerência de Configuração

[Git] Fundamentos Git

Profa. Lana Mesquita



Conteúdo

- Controle de Versão
- VCS
- Criação do Git
- Git
- Instalando o Git
- `git init`
- `git status`
- `git clone`
- `git remote`

Controle de versões (VCS)

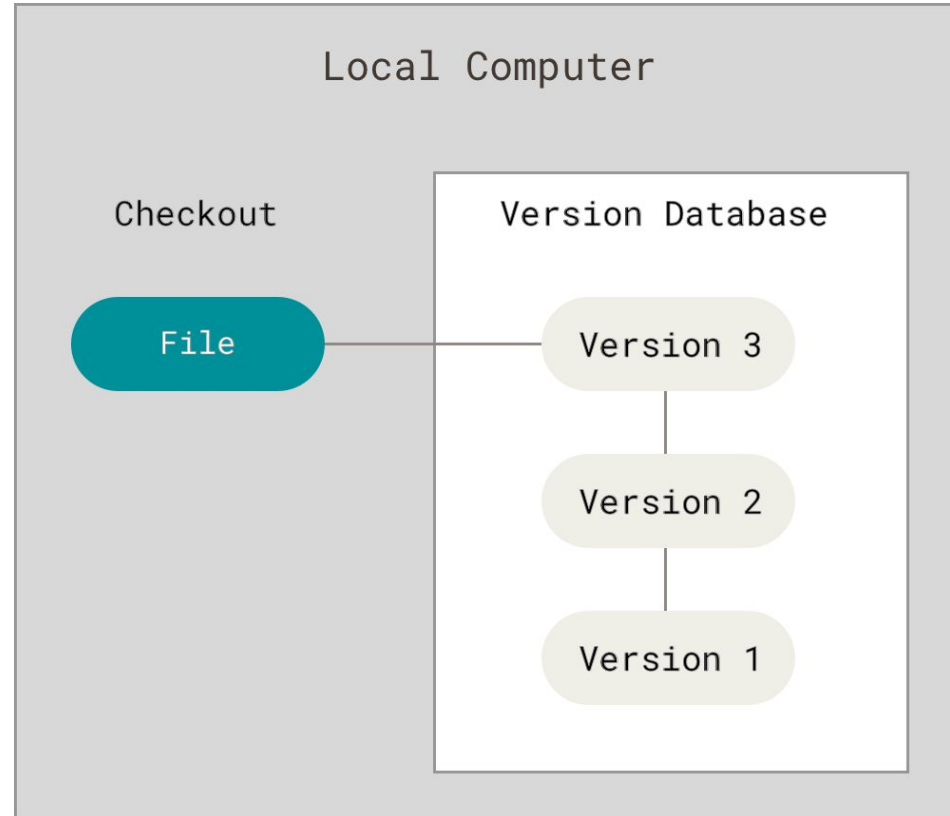
Controle de versão é um sistema que **registra alterações** em um arquivo ou conjunto de arquivos ao longo do tempo para que você possa lembrar versões específicas mais tarde.

Utilidade do VCS

Um VCS permite:

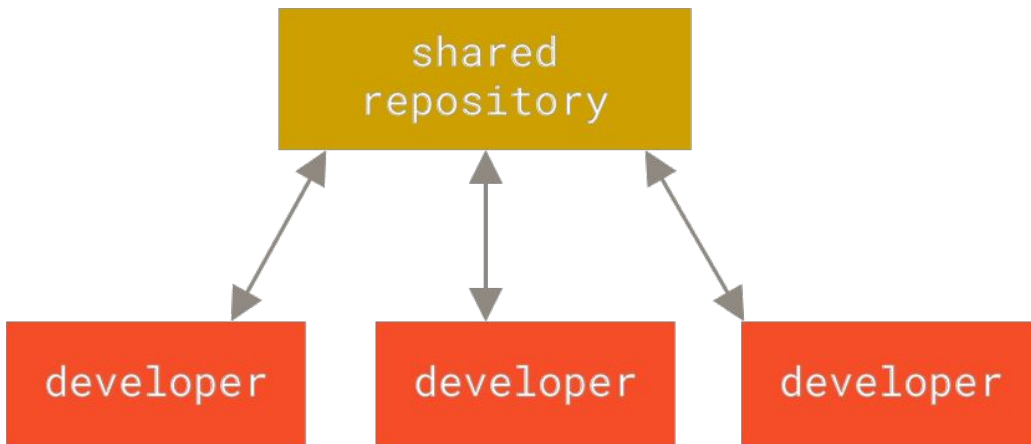
- reverter para estado anterior determinados arquivos ou um projeto inteiro;
- comparar as mudanças ao longo do tempo;
- ver quem modificou pela última vez algo que pode estar causando um problema;
- realizar uma cópia do projeto;
- trabalhar offline com um repositório do projeto em sua máquina;
- estragar tudo ou perder arquivos, já que é possível recuperá-los.

Utilidade do VCS



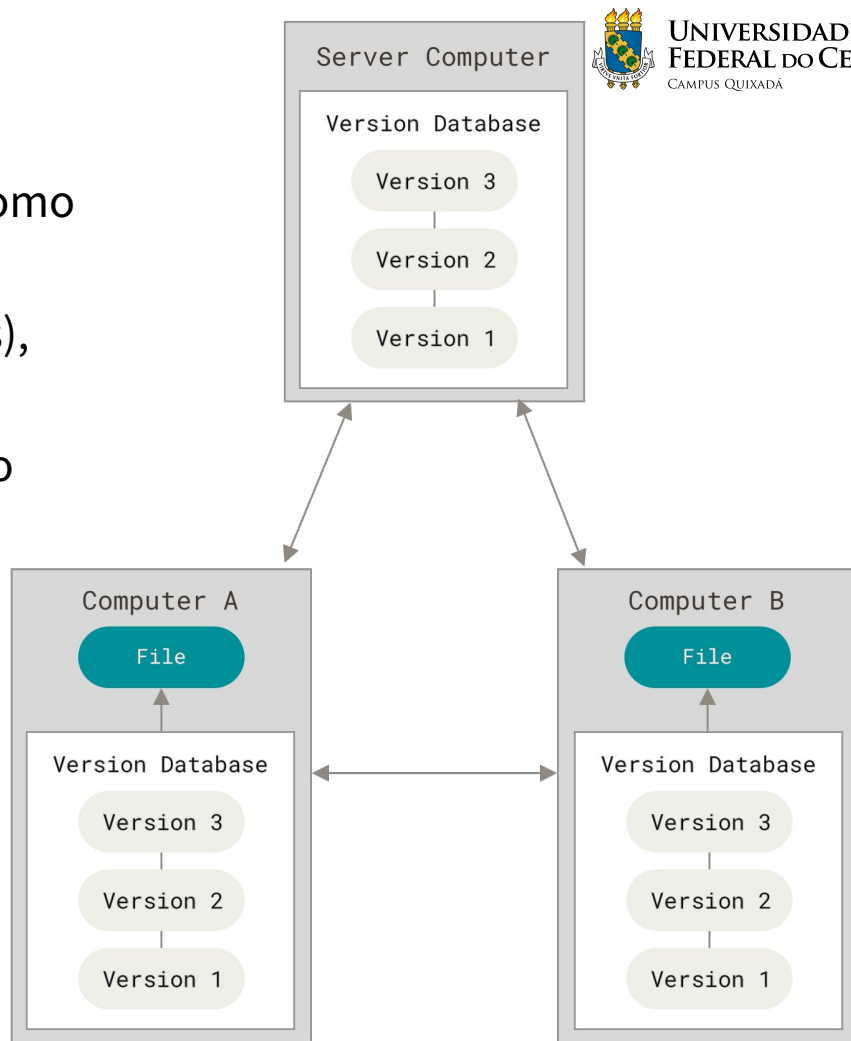
Sistemas Centralizados de Controle de Versão CVCS

CVS, Subversion e Perforce, têm um único servidor que contém todos os arquivos de controle de versão, e um número de clientes que usam arquivos a partir desse lugar central.

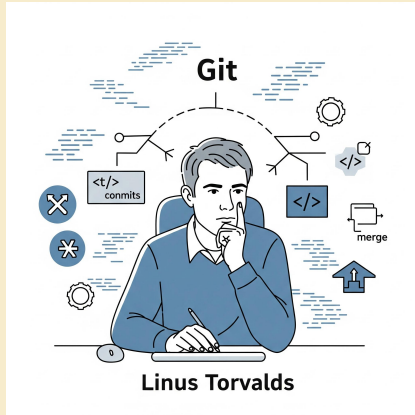


Sistemas Distribuídos de Controle de Versão DVCS

Em um DVCS (como **Git**, Mercurial, Bazaar ou Darcs), clientes não somente usam o estado mais recente dos arquivos: eles duplicam localmente o repositório completo.

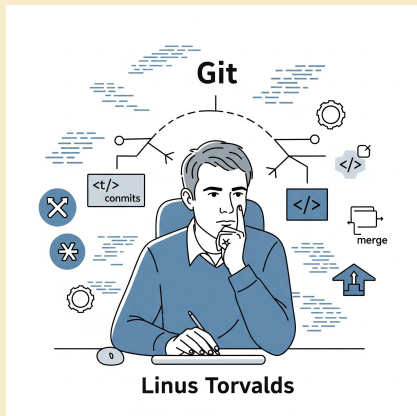


A Criação do Git



- O núcleo (kernel) do Linux é um projeto de código aberto com um escopo bastante grande.
- Na maior parte da vida da manutenção do núcleo o Linux (1991-2002), as mudanças no código eram compartilhadas como correções e arquivos.
- Em **2002**, o projeto do núcleo do Linux começou usar uma DVCS proprietária chamada BitKeeper.
- Em **2005** a ferramenta passou a ser paga e a comunidade decidiu não usar mais. (quem nunca? 🙄)

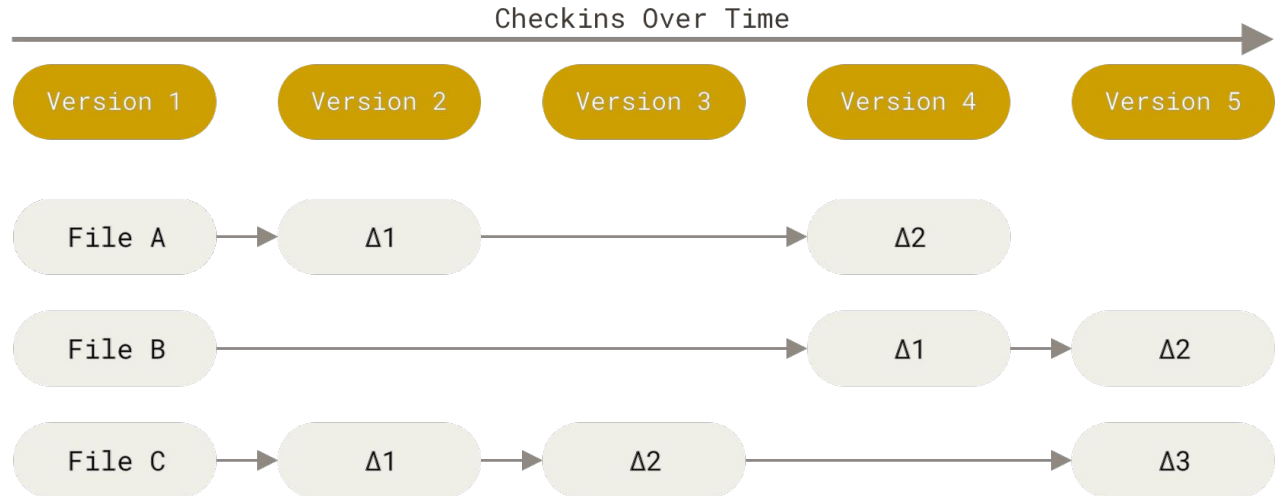
A Criação do Git



- A comunidade que desenvolvia o Linux (e especialmente Linus Torvalds, o criador do Linux) decidiram desenvolver a própria ferramenta.
- Foco em
 - Velocidade
 - Projeto simples
 - Forte suporte para desenvolvimento não-linear
 - Completamente distribuído
 - Capaz de lidar com projetos grandes como o núcleo o Linux com eficiência (velocidade e tamanho dos dados)
- O Git evoluiu ao longo dos anos para ser fácil de usar e eficiente

Outros VCSs

Outros VCSs como CVS, Subversion, Perforce, Bazaar, e outros, tratam a informação como um conjunto de arquivos e as mudanças feitas em cada arquivo ao longo do tempo.

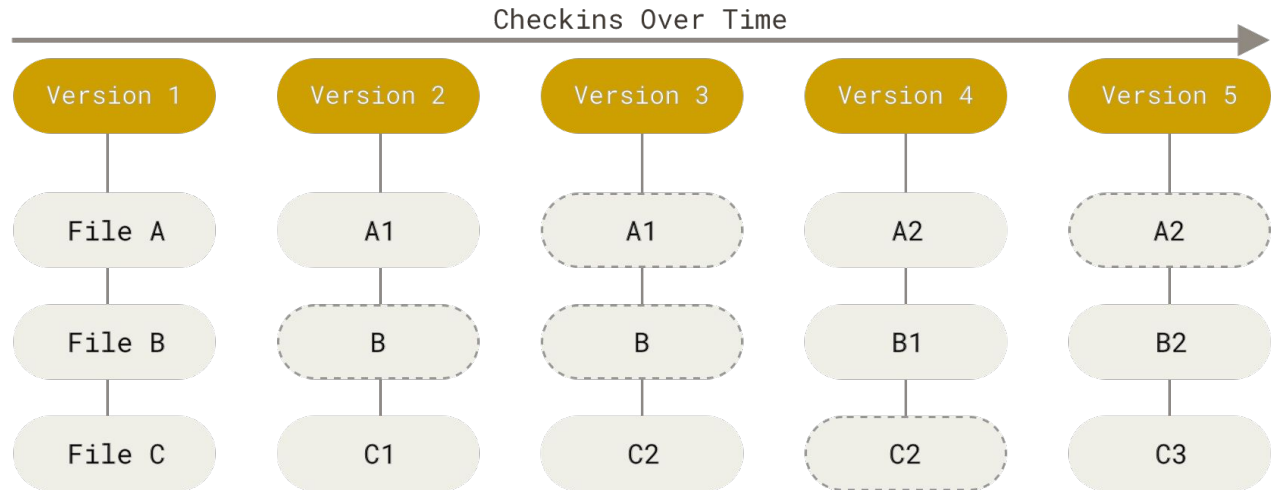


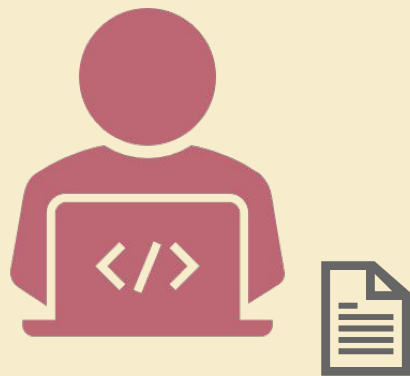
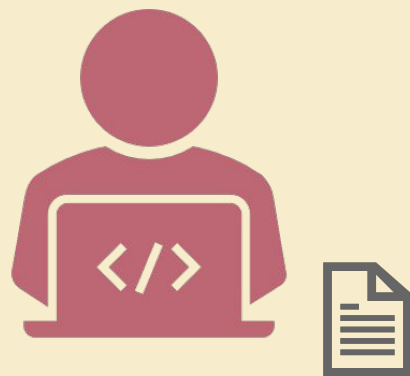
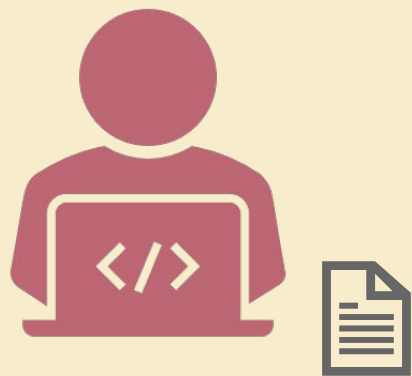
Git

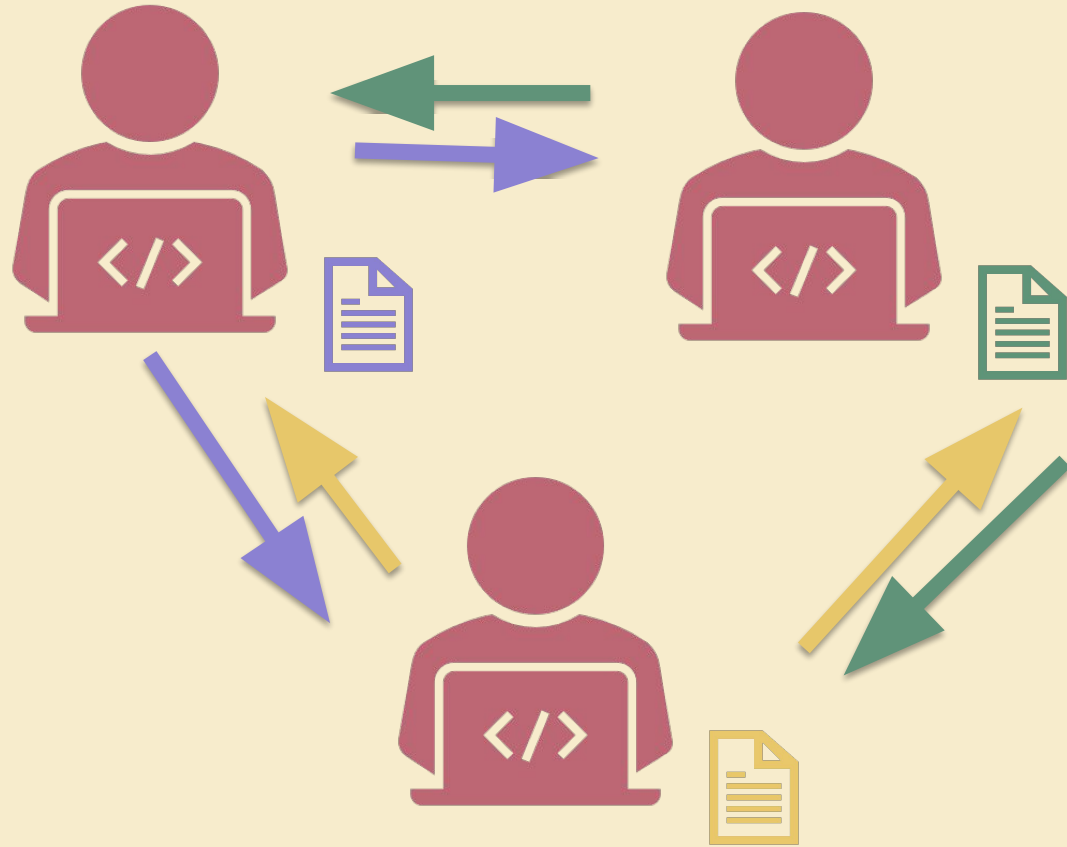
Para ser eficiente, se os arquivos não foram alterados, o Git não armazena o arquivo novamente, apenas um link para o arquivo idêntico anterior já armazenado.

O Git trata seus dados mais como um fluxo do estado dos arquivos.

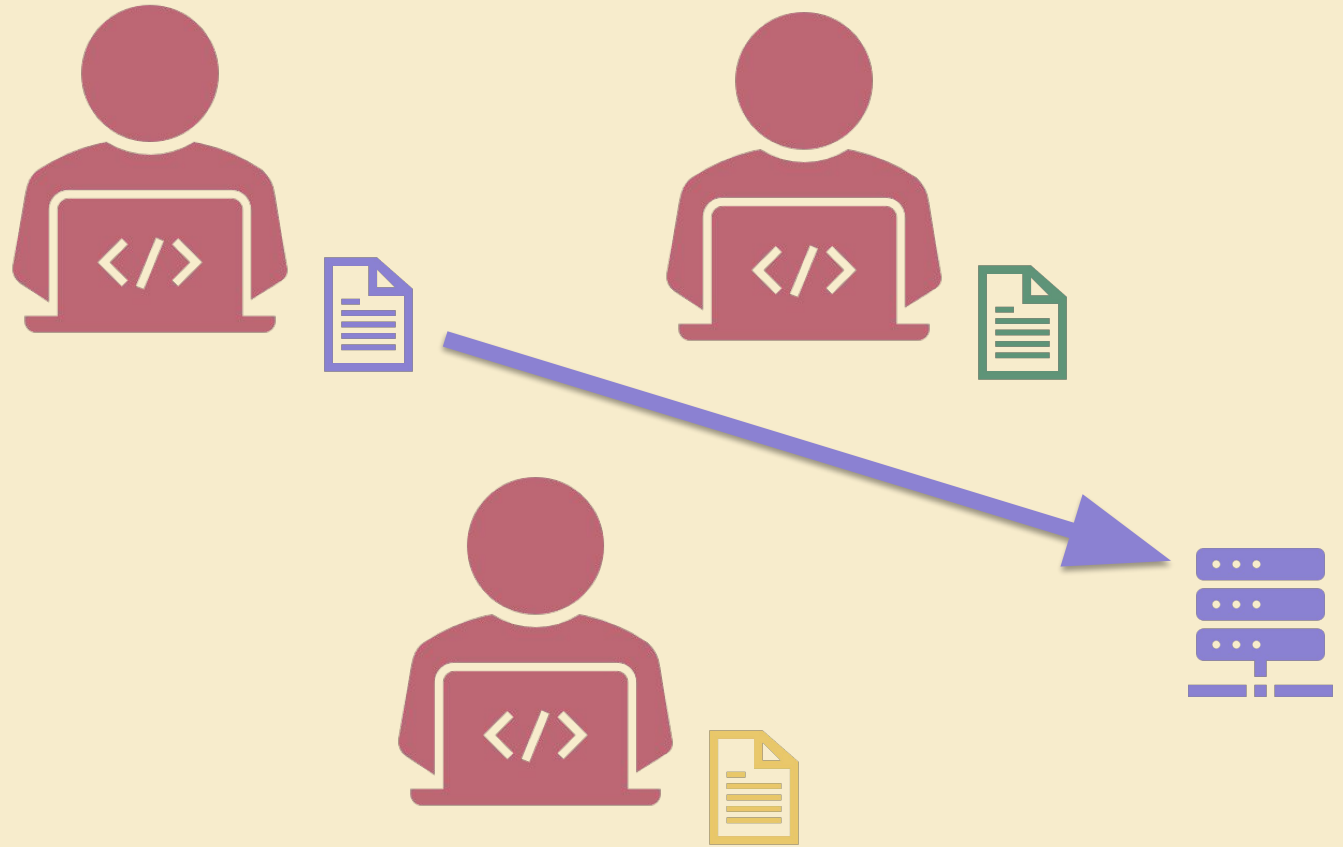
Git é um sistema que funciona como miniaturas dos arquivos guardando as **mudanças** que ocorreram entre versões. É uma sequência de **snapshots**.



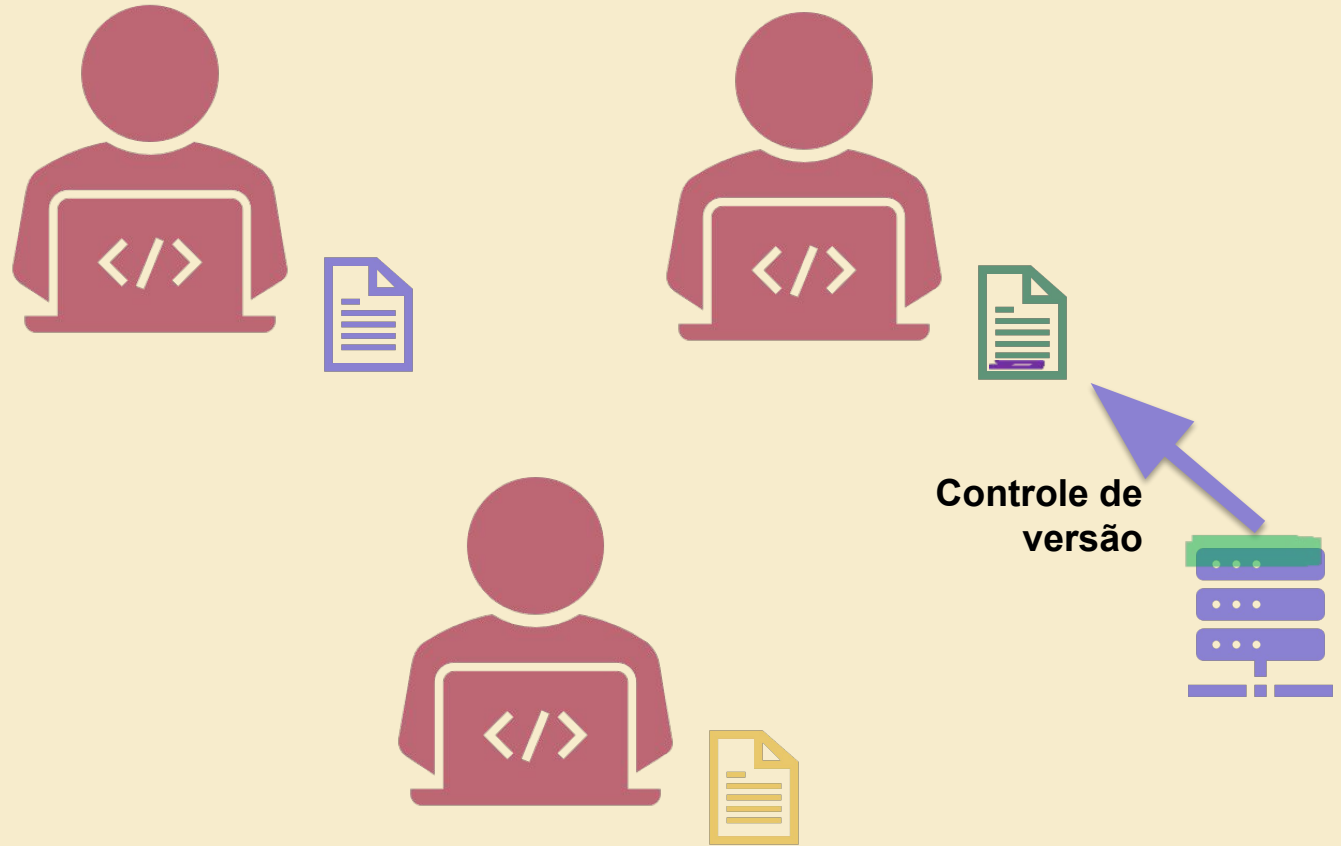




?







Integridade no Git

- Garante **integridade**: Tudo no Git passa por uma soma de verificações (checksum) antes de ser armazenado e é referenciado por esse checksum (hash *SHA-1*, exemplo abaixo).

24b9da6552252987aa493b52f8696cd6d3b00373

- Você vai ver esses valores de hash em todo o lugar no Git porque ele os usa com frequência.
- O Git armazena tudo em seu banco de dados não pelo nome do arquivo, mas pelo valor de hash do seu conteúdo.

Git



- Sistema de controle de versões
- Distribuído
- Rápido e leve
- Design simples
- Suporte robusto a desenvolvimento não linear
- Capaz de lidar eficientemente com grandes projetos como o kernel do Linux

A distribuição Git é escrita em C, Shell, Perl e Tcl

<https://github.com/git/git>

Serviços de hospedagem de projetos com controle de versionamento



software livre





Serviços de hospedagem de projetos com controle de versionamento

- GitHub é um **Serviço de Web Hosting Compartilhado** para projetos que usam o controle de versionamento Git.
- É escrito em Ruby on Rails pelos desenvolvedores da Logical Awesome (Chris Wanstrath, PJ Hyett e Tom Preston - Wernder).
- O GitHub possui planos comerciais e gratuitos para projetos de código aberto.
- Possui funcionalidades de uma rede social como feeds, followers, wiki e um gráfico que mostra como os desenvolvedores trabalham as versões de seus repositórios.

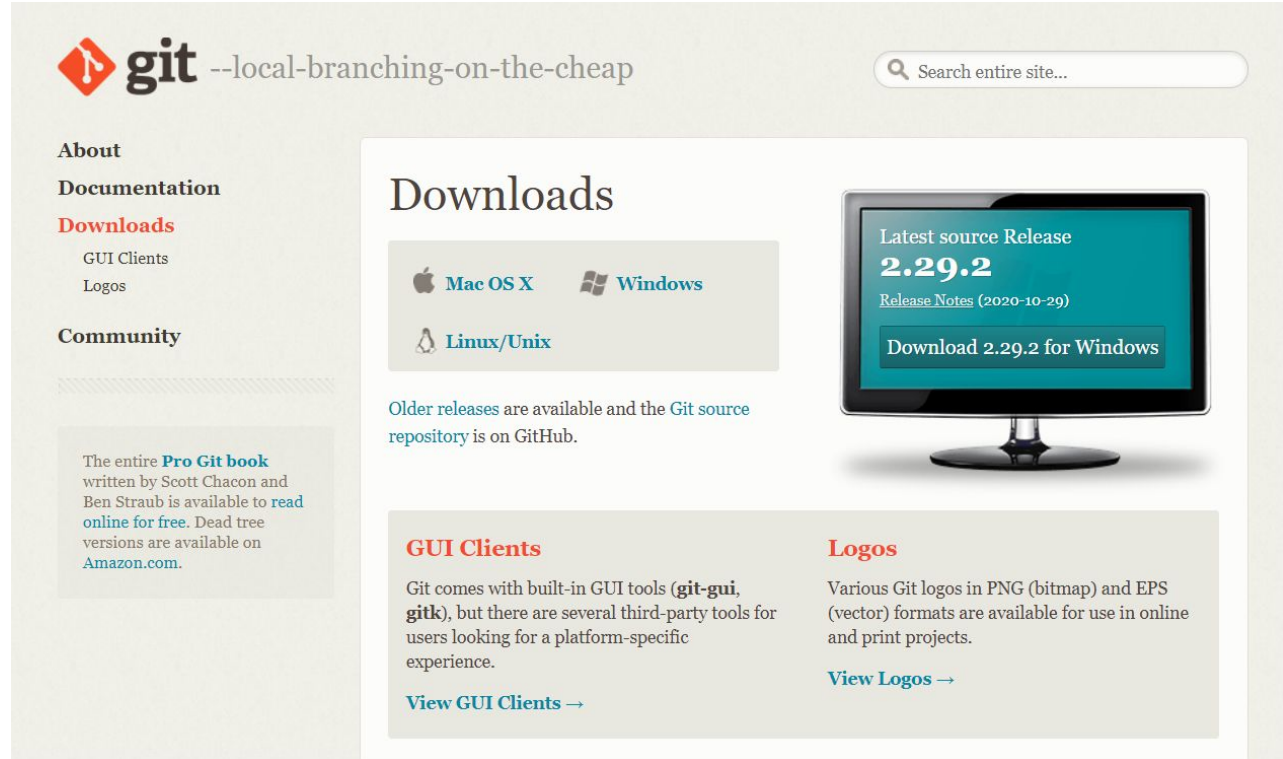
Vamos instalar o Git

Instalação Git

<https://git-scm.com/downloads>

Para **Linux**: algumas distribuições já vêm com o Git instalado, então é só abrir o Terminal e digitar "git" para verificar isto.

Se ele não estiver instalado, use o gerenciador de pacotes da sua distribuição, por exemplo o **APT** para Ubuntu ou derivados de Debian, o DNF para Fedora. (mais detalhes veja: <https://github.com/git-guides/install-all-git>)



The screenshot shows the Git website homepage. At the top, the Git logo is followed by the tagline "--local-branching-on-the-cheap". A search bar is located in the top right corner. The main navigation menu includes links for "About", "Documentation", "Downloads" (highlighted in red), and "Community". Below the navigation menu, there is a section for "The entire Pro Git book" written by Scott Chacon and Ben Straub, available to read online for free. The "Downloads" section features a large "Downloads" heading and three download options: "Mac OS X", "Windows", and "Linux/Unix". A monitor graphic displays the "Latest source Release 2.29.2" with a "Download 2.29.2 for Windows" button. A note mentions that older releases are available and the Git source repository is on GitHub. The "GUI Clients" section lists built-in tools (git-gui, gitk) and third-party tools. The "Logos" section mentions various Git logos in PNG and EPS formats. A "View GUI Clients" link is provided at the bottom of the GUI Clients section.

git --local-branching-on-the-cheap

Search entire site...

About
Documentation
Downloads
GUI Clients
Logos
Community

The entire **Pro Git book** written by Scott Chacon and Ben Straub is available to [read online for free](#). Dead tree versions are available on [Amazon.com](#).

Downloads

Older releases are available and the [Git source repository](#) is on GitHub.

Latest source Release
2.29.2
[Release Notes \(2020-10-29\)](#)
[Download 2.29.2 for Windows](#)

GUI Clients

Git comes with built-in GUI tools (**git-gui**, **gitk**), but there are several third-party tools for users looking for a platform-specific experience.

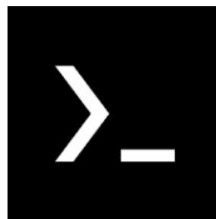
[View GUI Clients →](#)

Logos

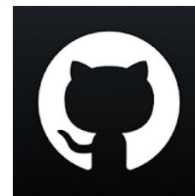
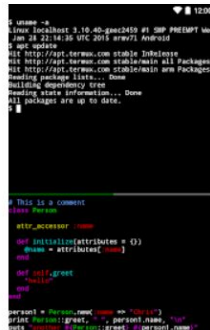
Various Git logos in PNG (bitmap) and EPS (vector) formats are available for use in online and print projects.

[View Logos →](#)

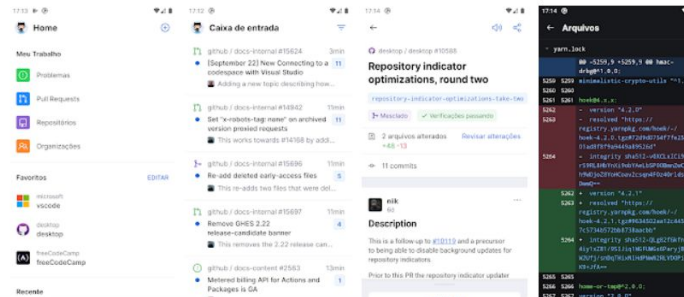
Não tenho computador, o que fazer?

★ ★ ★ ★ ★ 88.905 

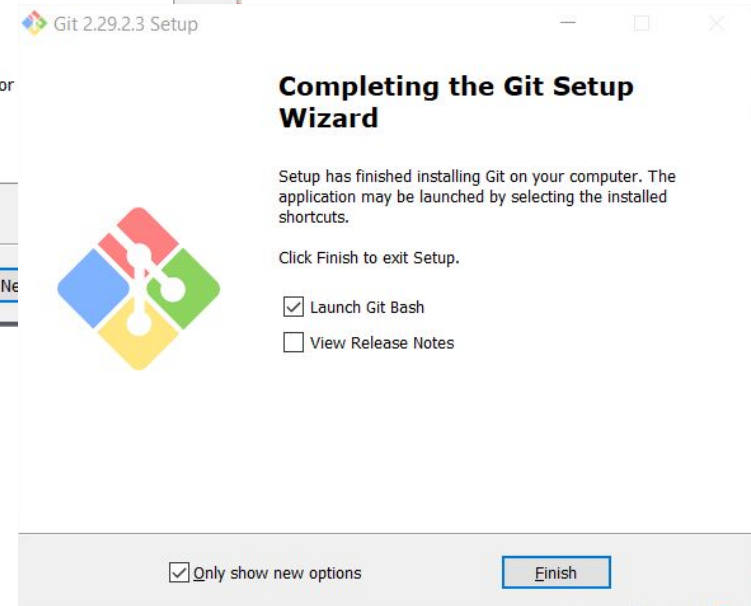
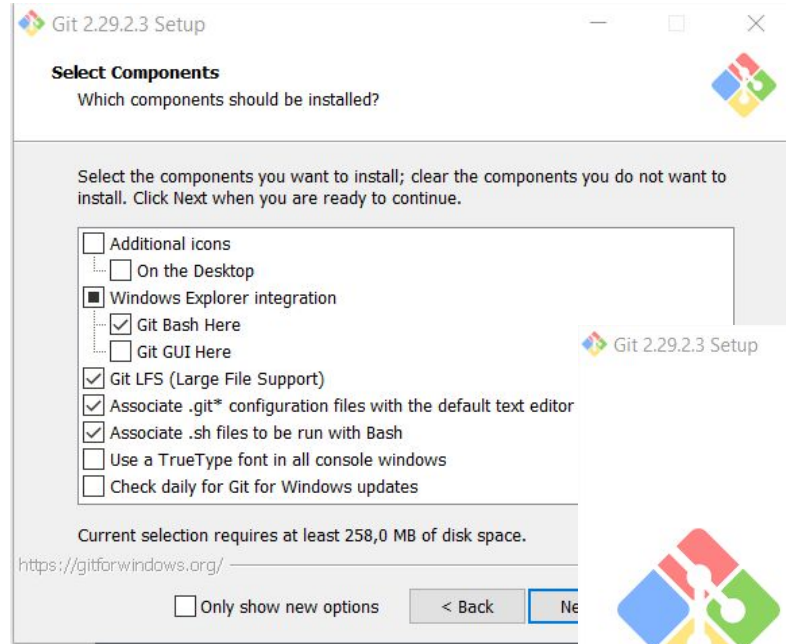
 Este app é compatível com seu dispositivo

 Adicionar à lista de desejos★★★★★ 12.487

 Este app é compatível com seu dispositivo.

 Adicionar à lista de desejos

Instalação Git



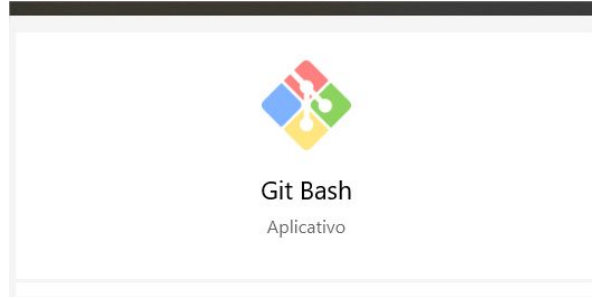
Instalação Git

Git Bash

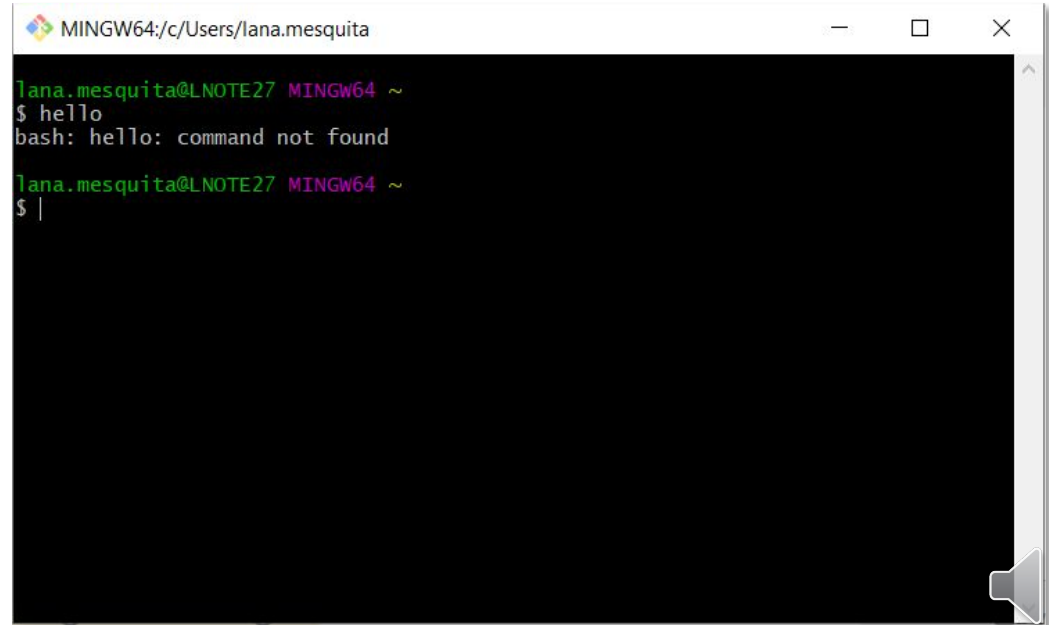
Se usar o Prompt
do Windows (cmd)



Prompt de Comando
Aplicativo



Coloquem pra fora o
Linux que há em vocês!



```
MINGW64; c:/Users/lana.mesquita

lana.mesquita@LNOTE27 MINGW64 ~
$ hello
bash: hello: command not found

lana.mesquita@LNOTE27 MINGW64 ~
$ |
```



git help

<https://git-scm.com/book/pt-br/v2/Come%C3%A7ando-Pedindo-Ajudade>

Pedindo ajuda

Há três formas de acessar a página do manual de ajuda (manpage) para qualquer um dos comandos Git:

```
$ git help <verb>
```

```
$ git <verb> --help
```

```
$ man git-<verb>
```

Exemplo: `$ git help config`

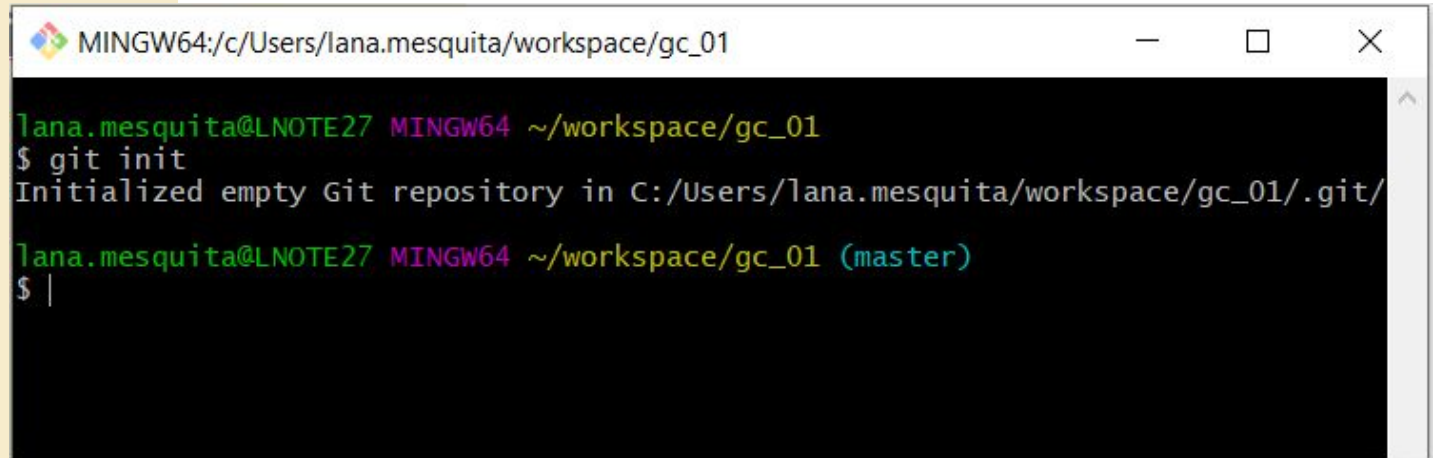
Estes comandos podem ser acessados de qualquer lugar, mesmo offline (💡 dica boa para provas, desafios de trabalho e até concursos).

git init

<https://github.com/git-guides/git-init>

Instalação Git

Git init



```
MINGW64:/c/Users/lana.mesquita/workspace/gc_01

lana.mesquita@LNOTE27 MINGW64 ~/workspace/gc_01
$ git init
Initialized empty Git repository in C:/Users/lana.mesquita/workspace/gc_01/.git/

lana.mesquita@LNOTE27 MINGW64 ~/workspace/gc_01 (master)
$ |
```

Instalação Git

Verificando a
versão

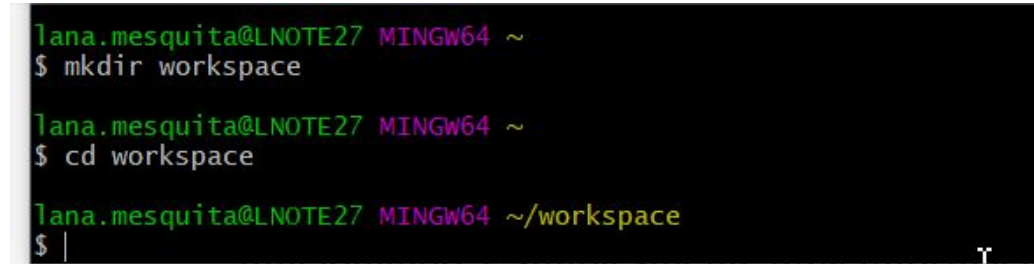


```
MINGW64:/c/Users/lana.mesquita

lana.mesquita@LNOTE27 MINGW64 ~
$ git version
git version 2.29.2.windows.3

lana.mesquita@LNOTE27 MINGW64 ~
$ |
```

Criando uma
pasta de trabalho



```
lana.mesquita@LNOTE27 MINGW64 ~
$ mkdir workspace

lana.mesquita@LNOTE27 MINGW64 ~
$ cd workspace

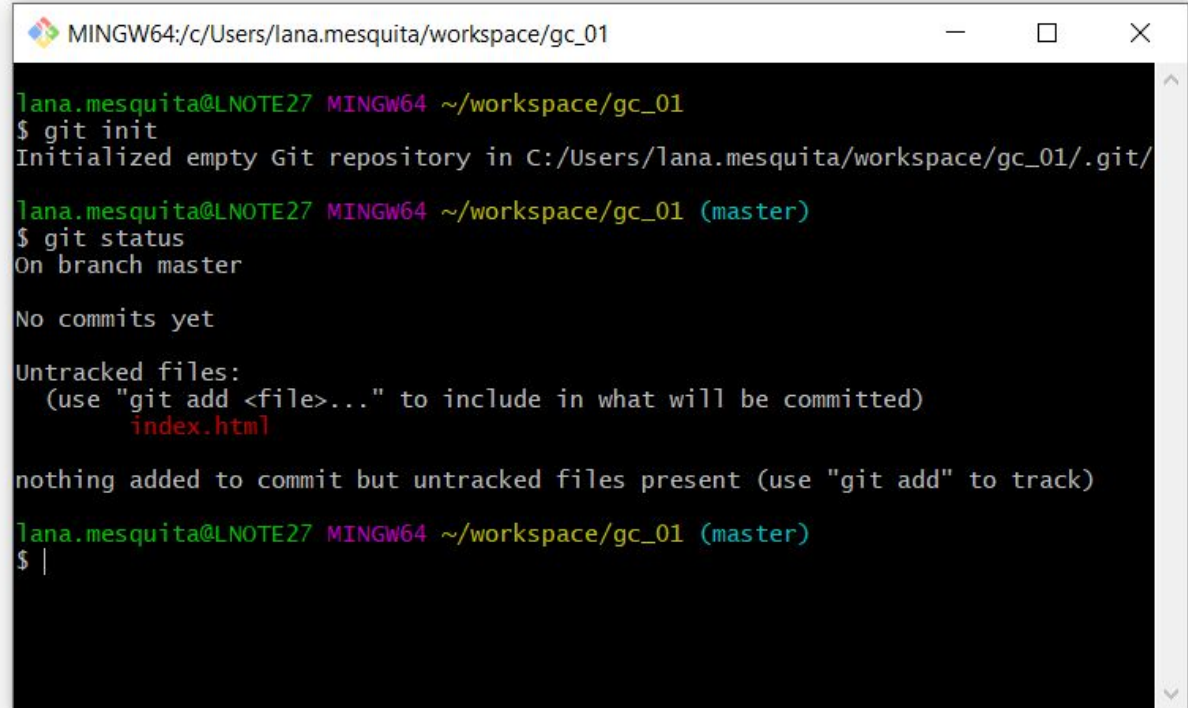
lana.mesquita@LNOTE27 MINGW64 ~/workspace
$ |
```


git status

<https://github.com/git-guides/git-status>

Instalação Git

Git status



```
MINGW64:/c:/Users/lana.mesquita/workspace/gc_01

lana.mesquita@LNOTE27 MINGW64 ~/workspace/gc_01
$ git init
Initialized empty Git repository in C:/Users/lana.mesquita/workspace/gc_01/.git/

lana.mesquita@LNOTE27 MINGW64 ~/workspace/gc_01 (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        index.html

nothing added to commit but untracked files present (use "git add" to track)

lana.mesquita@LNOTE27 MINGW64 ~/workspace/gc_01 (master)
$ |
```

git clone

<https://github.com/git-guides/git-clone>

Instalação Git

Clonar um
Repositório
em vez de criar um
local

obtenha um repositório

crie uma cópia de trabalho em um repositório local executando o
comando

```
git clone /caminho/para/o/repositório
```

quando usar um servidor remoto, seu comando será

```
git clone usuário@servidor:/caminho/para/o/repositório
```

git remote

<https://github.com/git-guides/git-remote>



Criando um repositório remoto

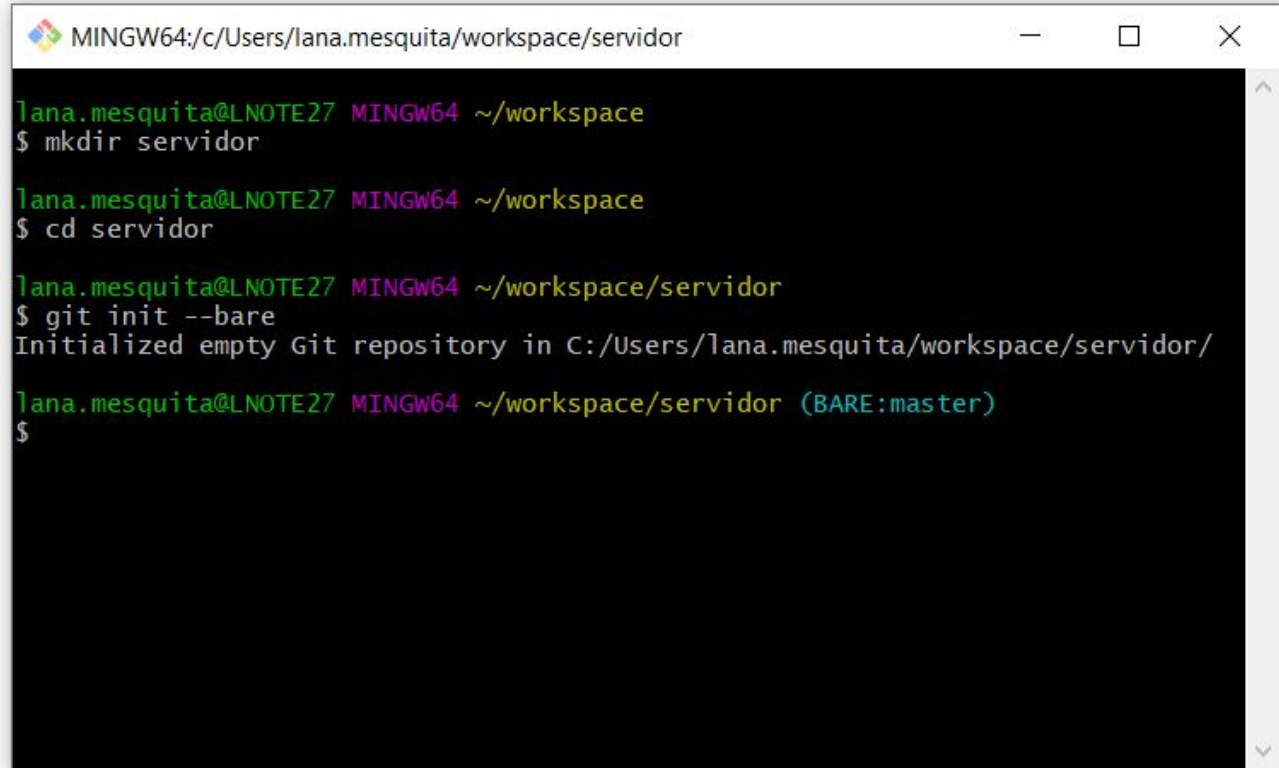
- Um repositório remoto pode ser:
 - Na sua própria máquina
C:/Users/lana.mesquita/workspace/servidor
 - Em um servidor na rede interna
https://192.168.0.181:8080
http://localhost:3000
 - Em um servidor na internet
https://github.com/user/repo.git



Criando um repositório remoto

comandos básicos de navegação em bash:
mkdir: criar pasta
cd: acessar uma pasta

git init --bare



```
MINGW64:/c/Users/lana.mesquita/workspace/servidor

lana.mesquita@LNOTE27 MINGW64 ~/workspace
$ mkdir servidor

lana.mesquita@LNOTE27 MINGW64 ~/workspace
$ cd servidor

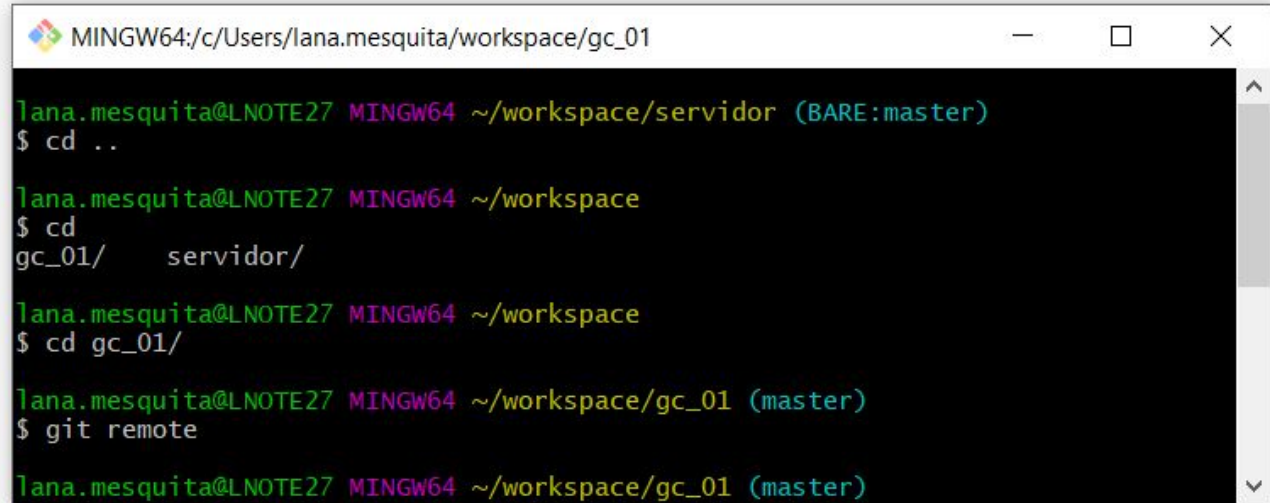
lana.mesquita@LNOTE27 MINGW64 ~/workspace/servidor
$ git init --bare
Initialized empty Git repository in C:/Users/lana.mesquita/workspace/servidor/

lana.mesquita@LNOTE27 MINGW64 ~/workspace/servidor (BARE:master)
$
```



Criando um repositório remoto

git remote



```
MINGW64:/c/Users/lana.mesquita/workspace/gc_01

lana.mesquita@LNOTE27 MINGW64 ~/workspace/servidor (BARE:master)
$ cd ..

lana.mesquita@LNOTE27 MINGW64 ~/workspace
$ cd
gc_01/      servidor/

lana.mesquita@LNOTE27 MINGW64 ~/workspace
$ cd gc_01/

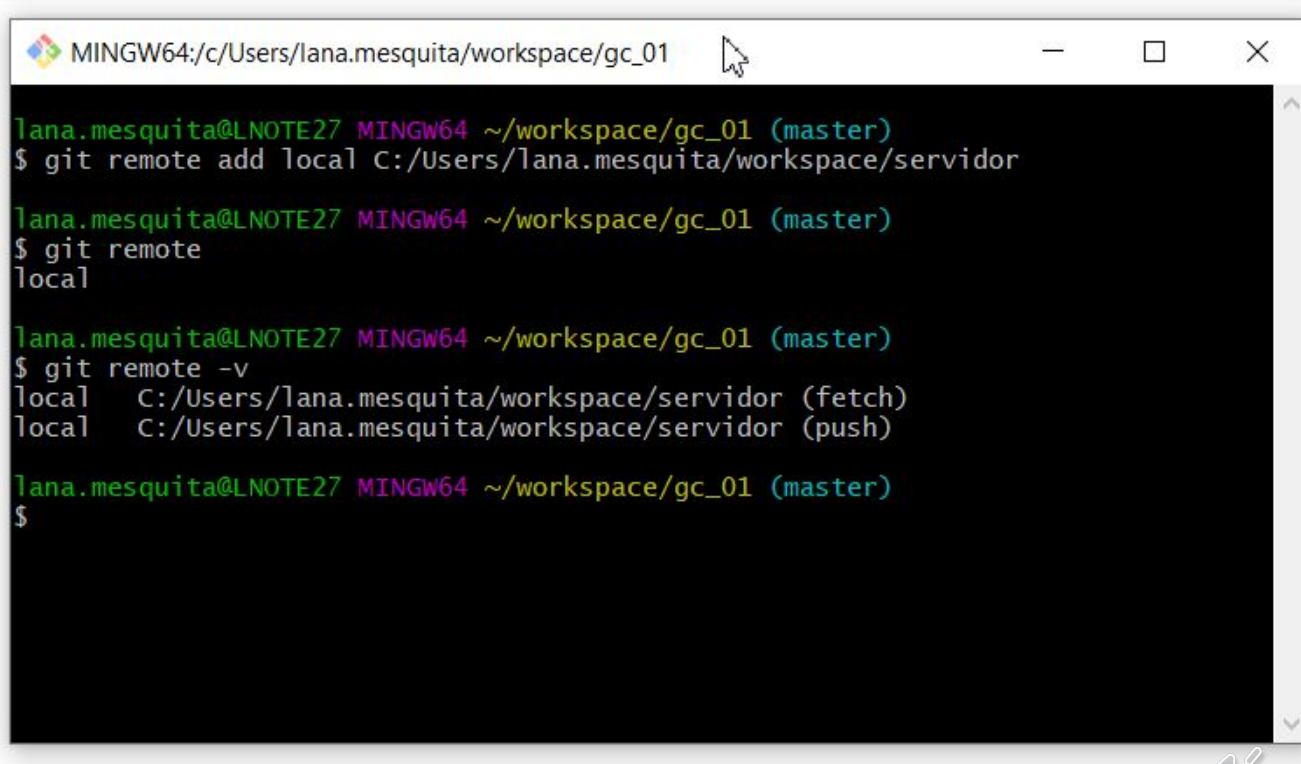
lana.mesquita@LNOTE27 MINGW64 ~/workspace/gc_01 (master)
$ git remote

lana.mesquita@LNOTE27 MINGW64 ~/workspace/gc_01 (master)
```



Criando um repositório remoto

git remote add
git remote -v



```
MINGW64:/c/Users/lana.mesquita/workspace/gc_01

lana.mesquita@LNOTE27 MINGW64 ~/workspace/gc_01 (master)
$ git remote add local C:/Users/lana.mesquita/workspace/servidor

lana.mesquita@LNOTE27 MINGW64 ~/workspace/gc_01 (master)
$ git remote
local

lana.mesquita@LNOTE27 MINGW64 ~/workspace/gc_01 (master)
$ git remote -v
local    C:/Users/lana.mesquita/workspace/servidor (fetch)
local    C:/Users/lana.mesquita/workspace/servidor (push)

lana.mesquita@LNOTE27 MINGW64 ~/workspace/gc_01 (master)
$
```



Revisão

- Sistemas de Controle de Versões (SCM) ajudam em
 - fluxo de desenvolvimento;
 - manter um histórico de alterações;
- O que é o Git e como instalá-lo
- Criação de um repositório Git usando **git clone**, que inicia e copia um repositório remoto, ou **git init**, que cria um repositório vazio.
- Como analisar o estado do repositório usando **git status**
- **git remote** pode ser usado para adicionar ou verificar o repositório remoto.

Dúvidas?
Mandem por Discord!



Até a próxima aula!

Lana Mesquita

